PROGRAMMING 621 ASSIGNMENT

Name and Surname: NHLANHLA ERIC MOYO
Student ITS No: 402104184
Qualification:DIT
RICHFIELD
Year of Study: 2026
Semester:2 RE-WRITE
Assignment due date: Date 21/09/26
Date submitted: Date 21/09/26

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

☑

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

☑

For essay-style assignments, please include the following sections:

- Table of Contents
- Introduction
- Main Body (with relevant subheadings)
- Conclusion
- References

☑

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

☑

Include a signed declaration of originality.
Richfield.ac.za

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.
Student Signature: Person Nhlexx
Date: Date 21/09/26

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

1.1.1 Bank Teller Authentication

Teller class definition (`Models.h`)

```
class Teller {
private:
string tellerId; // T001
string fullName;
string encryptedPassword;
string branchCode; // restricts teller to their branch
public:
Teller();
Teller(const string& id, const string& name, const string& pwd, const string& branch);

string getTellerId() const { return tellerId; }
string getFullName() const { return fullName; }
string getEncryptedPassword() const { return encryptedPassword; }
string getBranchCode() const { return branchCode; }

void setPassword(const string& pwd) { encryptedPassword = Utils::encryptText(pwd); }
bool checkPassword(const string& pwd) const { return Utils::verifyText(pwd, encryptedPassword); }

void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};
```

Encryption algorithm (`Utils.cpp`)

```
*/
string encryptText(const string& plain) {
static const char* HEX = "0123456789ABCDEF";
string out;
out.reserve(plain.size() * 2);
for (size_t i = 0; i < plain.size(); ++i) {
unsigned char c = (unsigned char)plain[i];
c = (unsigned char)(c ^ XOR_KEY);
c = (unsigned char)((c + SHIFT) & 0xFF);
out += HEX[(c >> 4) & 0x0F];
```

```
out += HEX[c & 0x0F];
}
return out;
}
```

Teller login (`BankSystem.cpp`)

```
bool BankSystem::loginTeller(const string& id, const string& pwd, Teller** outTeller) {
for (size_t i = 0; i < tellers.size(); ++i) {
if (Utils::toUpper(tellers[i].getTellerId()) == Utils::toUpper(id)) {
if (tellers[i].checkPassword(pwd)) {
*outTeller = &tellers[i];
return true;
}
return false;
}
}
return false;
}

void BankSystem::tellerLogin() {
Utils::clearScreen();
printHeader("BANK TELLER LOGIN");

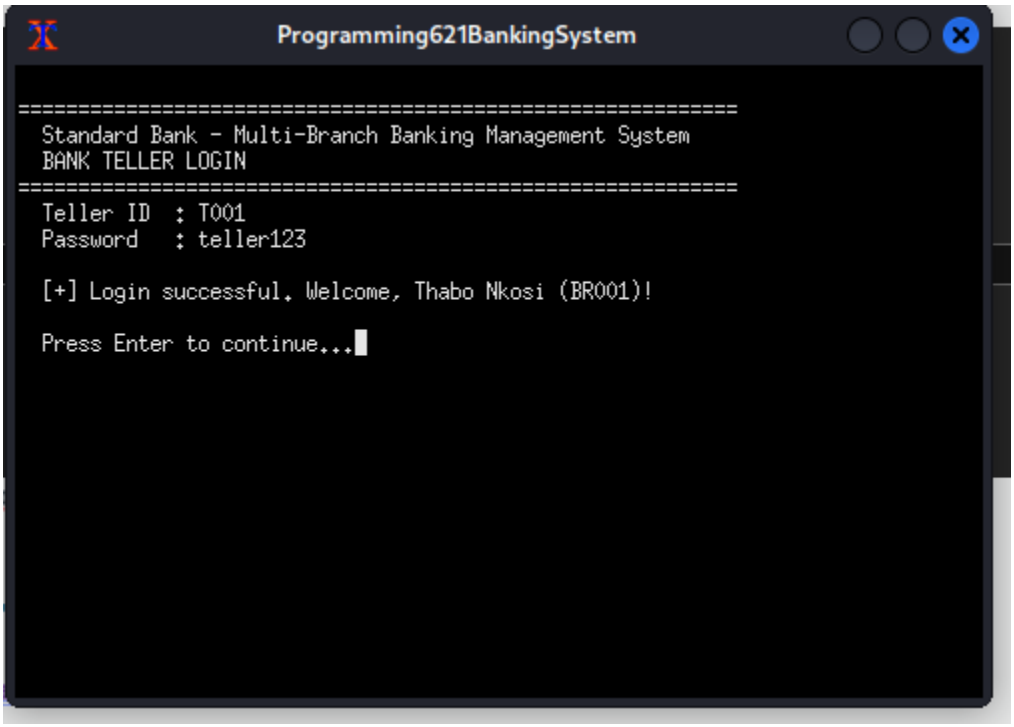
int attempts = 0;
while (attempts < maxLoginAttempts) {
string id = Utils::readLine(" Teller ID : ");
string pwd = Utils::readLine(" Password : ");

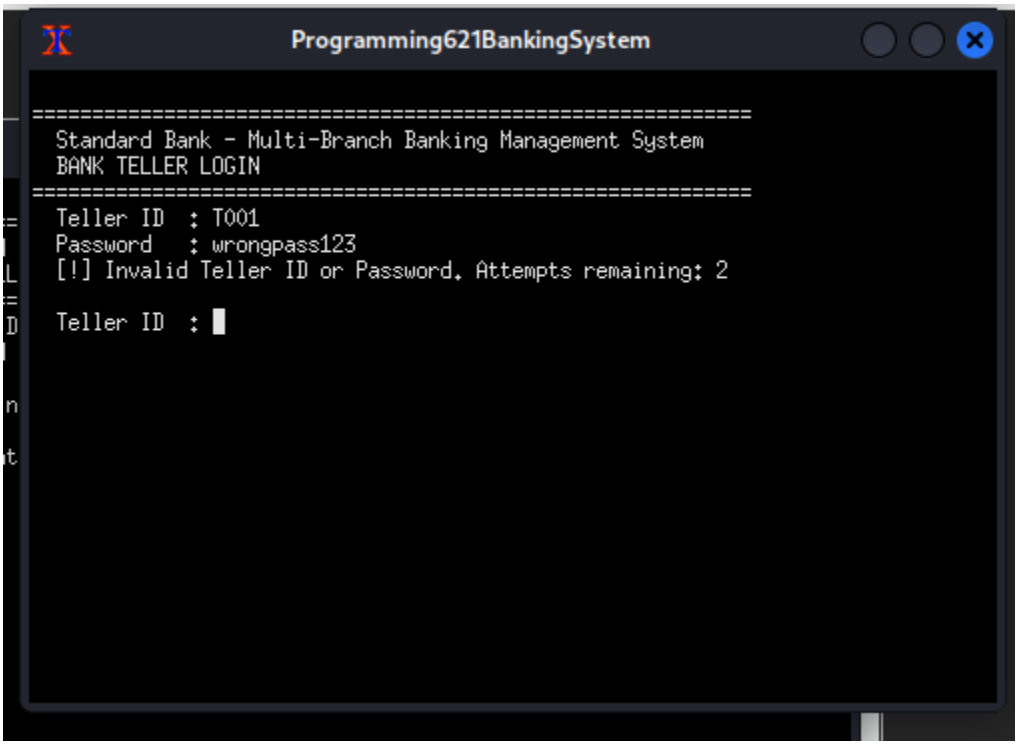
Teller* t = 0;
if (loginTeller(id, pwd, &t)) {
currentTeller = t;
cout << "\n [+] Login successful. Welcome, " << t->getFullName()
<< " (" << t->getBranchCode() << ")!\n";
Utils::pause();
tellerMenu();
return;
}
```

```
attempts++;
cout << " [!] Invalid Teller ID or Password. Attempts remaining: "
<< (maxLoginAttempts - attempts) << "\n\n";
}

cout << "\n [!] Too many failed login attempts. Returning to main menu.\n";
Utils::pause();
}
```

Output




Programming621BankingSystem
Standard Bank - Multi-Branch Banking Management System
BANK TELLER LOGIN
Teller ID : T001
Password : wrongpass123
[!] Invalid Teller ID or Password. Attempts remaining: 2
Teller ID :

1.1.2 Customer PIN-based Authentication

Customer PIN field (`Models.h`)

```
class Customer {
private:
string customerId; // CUST-0001
string firstName;
string surname;
string idNumber; // 13 digit SA ID
string contactNumber; // 10 digits
string email;
string address;
string dateOfBirth; // DD/MM/YYYY
string encryptedPin; // encrypted 5-digit PIN
int failedAttempts;
bool locked;
vector<Account*> accounts;

public:
Customer();
Customer(const string& custId, const string& fname, const string& sname,
const string& idNo, const string& contact, const string& mail,
const string& addr, const string& dob, const string& pin);
~Customer();

/* getters */
string getCustomerId() const { return customerId; }
string getFirstName() const { return firstName; }
string getSurname() const { return surname; }
string getFullName() const { return firstName + " " + surname; }
string getIdNumber() const { return idNumber; }
string getContactNumber() const { return contactNumber; }
string getEmail() const { return email; }
string getAddress() const { return address; }
string getDateOfBirth() const { return dateOfBirth; }
string getEncryptedPin() const { return encryptedPin; }
int getFailedAttempts() const { return failedAttempts; }
bool isLocked() const { return locked; }

vector<Account*>& getAccounts() { return accounts; }
```

```
/* setters */
void setEncryptedPin(const string& p) { encryptedPin = p; }
void setFailedAttempts(int n) { failedAttempts = n; }
void setLocked(bool l) { locked = l; }
void addAccount(Account* a) { accounts.push_back(a); }

Account* findAccount(const string& accNo) const;
double totalBalance() const;

void display() const;
void displayAccounts() const;

void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};
```

Customer login with lockout (BankSystem.cpp)

```
void BankSystem::customerLogin() {
Utils::clearScreen();
printHeader("CUSTOMER SELF-SERVICE LOGIN");

string accNo = Utils::toUpper(Utils::readLine(" Account Number (ACC-BRXXX-XXXXX): "));

Account* acc = 0;
Customer* cust = findCustomerByAccount(accNo, &acc);

if (!cust || !acc) {
cout << "\n [!] Account not found. Please check the account number.\n";
Utils::pause();
return;
}

if (cust->isLocked()) {
cout << "\n [!] This account is temporarily LOCKED due to too many failed login attempts.\n";
cout << " Please contact your branch to have it unlocked.\n";
Utils::pause();
return;
}
```

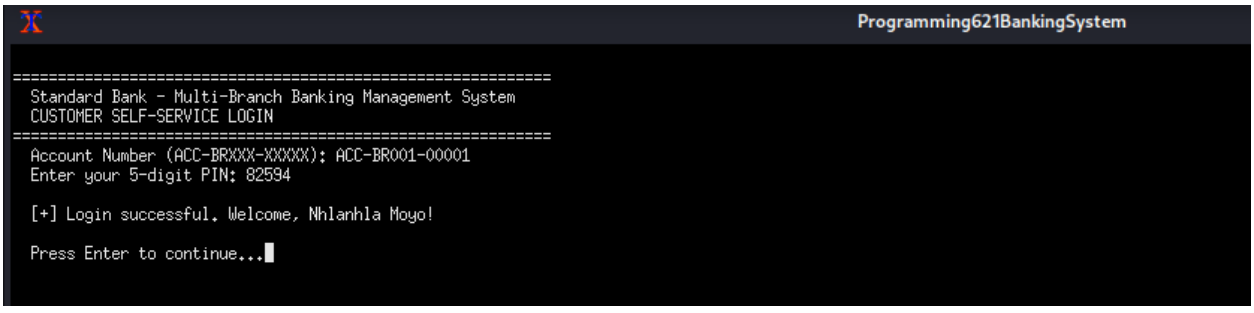
```
int attempts = 0;
while (attempts < maxLoginAttempts) {
string pin = Utils::readLine(" Enter your 5-digit PIN: ");

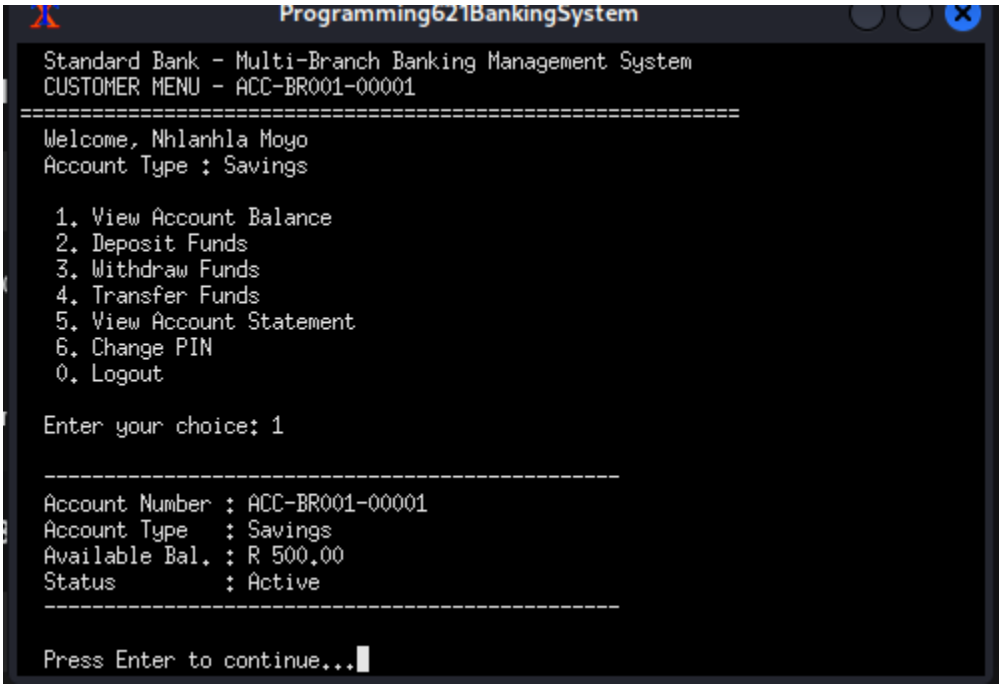
if (Utils::verifyText(pin, cust->getEncryptedPin())) {
cust->setFailedAttempts(0);
saveCustomers();
currentCustomer = cust;
currentAccountNumber = accNo;
cout << "\n [+] Login successful. Welcome, " << cust->getFullName() << "!\n";
Utils::pause();
customerMenu();
return;
}

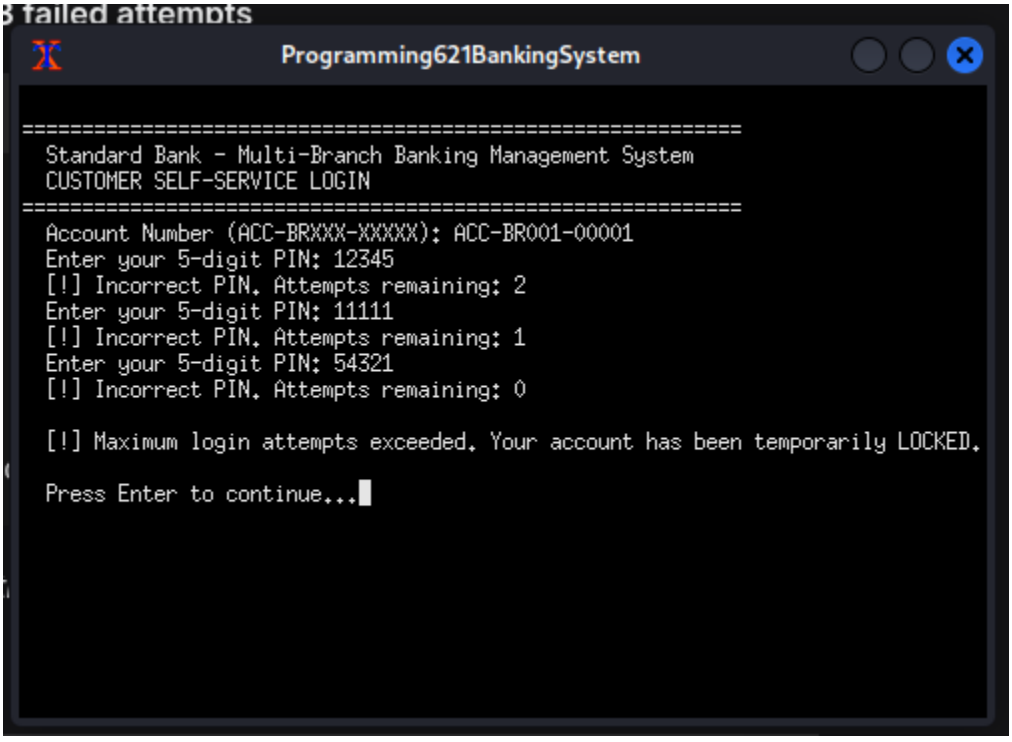
attempts++;
cust->setFailedAttempts(attempts);
saveCustomers();
cout << " [!] Incorrect PIN. Attempts remaining: "
<< (maxLoginAttempts - attempts) << "\n";
}

cust->setLocked(true);
saveCustomers();
cout << "\n [!] Maximum login attempts exceeded. Your account has been temporarily LOCKED.\n";
Utils::pause();
}
```

OUTPUT




Programming621BankingSystem
Standard Bank - Multi-Branch Banking Management System
CUSTOMER MENU - ACC-BR001-00001
Welcome, Nhlanhla Moyo
Account Type : Savings
1. View Account Balance
2. Deposit Funds
3. Withdraw Funds
4. Transfer Funds
5. View Account Statement
6. Change PIN
0. Logout
Enter your choice: 1
Account Number : ACC-BR001-00001
Account Type : Savings
Available Bal. : R 500.00
Status : Active
Press Enter to continue...


3 failed attempts
Programming621BankingSystem
Standard Bank - Multi-Branch Banking Management System
CUSTOMER SELF-SERVICE LOGIN
Account Number (ACC-BRXXX-XXXXX): ACC-BR001-00001
Enter your 5-digit PIN: 12345
[!] Incorrect PIN. Attempts remaining: 2
Enter your 5-digit PIN: 11111
[!] Incorrect PIN. Attempts remaining: 1
Enter your 5-digit PIN: 54321
[!] Incorrect PIN. Attempts remaining: 0
[!] Maximum login attempts exceeded. Your account has been temporarily LOCKED.
Press Enter to continue...

1.2 Customer Account Management [20 Marks]

1.2.1 Account Registration (Teller Function)

Validation helpers (Utils.cpp)

```
bool isAllDigits(const string& s) {
if (s.empty()) return false;
for (size_t i = 0; i < s.size(); ++i)
if (!isdigit((unsigned char)s[i])) return false;
return true;
}

/* South African ID: 13 digits and must pass the Luhn check digit */
bool validateSAID(const string& id) {
if (id.size() != 13) return false;
if (!isAllDigits(id)) return false;

int sum = 0;
bool alternate = false;
for (int i = 12; i >= 0; --i) {
int d = id[i] - '0';
if (alternate) {
d *= 2;
if (d > 9) d -= 9;
}
sum += d;
alternate = !alternate;
}
return (sum % 10 == 0);
}

bool validatePhone(const string& phone) {
return phone.size() == 10 && isAllDigits(phone);
}

bool validateEmail(const string& email) {
size_t at = email.find('@');
if (at == string::npos || at == 0) return false;
size_t dot = email.find('.', at);
if (dot == string::npos || dot == at + 1 || dot == email.size() - 1) return false;
if (email.find(' ') != string::npos) return false;
```

```
return true;
}

bool validateDate(const string& date) {
if (date.size() != 10) return false;
if (date[2] != '/' || date[5] != '/') return false;

string dd = date.substr(0, 2);
string mm = date.substr(3, 2);
string yyyy = date.substr(6, 4);

if (!isAllDigits(dd) || !isAllDigits(mm) || !isAllDigits(yyyy)) return false;

int d = atoi(dd.c_str());
int m = atoi(mm.c_str());
int y = atoi(yyyy.c_str());

if (y < 1900 || y > 2100) return false;
if (m < 1 || m > 12) return false;

int daysInMonth[] = {31,28,31,30,31,30,31,31,30,31,30,31};
if (m == 2 && isLeapYear(y)) daysInMonth[1] = 29;

if (d < 1 || d > daysInMonth[m - 1]) return false;
return true;
}
```

Registration flow (BankSystem.cpp)

```
void BankSystem::registerCustomer() {
Utils::clearScreen();
printHeader("REGISTER NEW CUSTOMER ACCOUNT");
cout << " Branch: " << currentTeller->getBranchCode() << "\n\n";

try {
/* ---- personal details ---- */
string fname = Utils::readLine(" First Name : ");
if (fname.empty()) throw BankingException("First name is required.");

string sname = Utils::readLine(" Surname : ");
```

```
if (sname.empty()) throw BankingException("Surname is required.");

/* ---- SA ID number ---- */
string idNo;
while (true) {
idNo = Utils::readLine(" SA ID Number (13 digits) : ");
if (!Utils::validateSAID(idNo)) {
cout << " [!] Invalid SA ID number (must be 13 digits and pass the Luhn check).\n";
continue;
}
if (findCustomerByIdNumber(idNo) != 0) {
cout << " [!] A customer with this SA ID already exists.\n";
continue;
}
break;
}

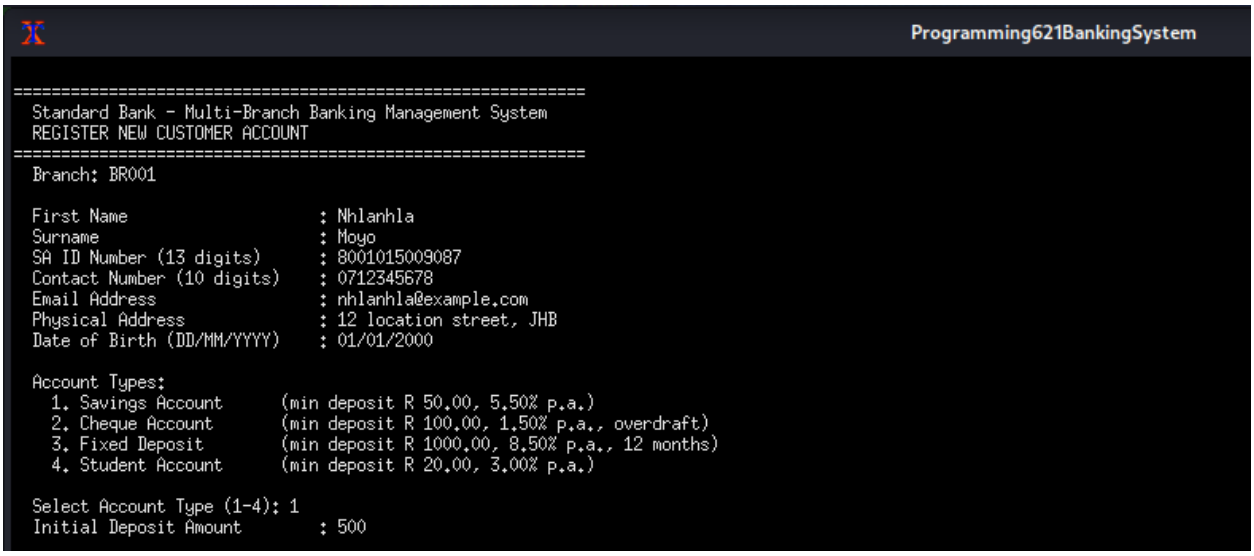
/* ---- contact number ---- */
string contact;
while (true) {
contact = Utils::readLine(" Contact Number (10 digits) : ");
if (!Utils::validatePhone(contact)) {
cout << " [!] Contact number must be exactly 10 digits.\n";
continue;
}
break;
}

/* ---- email ---- */
string mail;
while (true) {
mail = Utils::readLine(" Email Address : ");
if (!Utils::validateEmail(mail)) {
cout << " [!] Invalid email address (must contain '@' and a domain).\n";
continue;
}
break;
}

/* ---- address ---- */
string addr = Utils::readLine(" Physical Address : ");
if (addr.empty()) throw BankingException("Physical address is required.");
```

```
/* ---- date of birth ---- */
string dob;
while (true) {
dob = Utils::readLine(" Date of Birth (DD/MM/YYYY) : ");
if (!Utils::validateDate(dob)) {
cout << " [!] Invalid date. Use DD/MM/YYYY format.\n";
continue;
}
break;
}
```

OUTPUT





1.2.2 Account Types Implementation (Inheritance)

Base class with pure virtual functions (Models.h)

```
class Account {
protected:
string accountNumber;
string accountType;
double balance;
string dateOpened;
string status; // Active / Dormant
double totalDeposits;
double totalWithdrawals;

public:
Account();
Account(const string& accNo, const string& type, double initialDeposit, const string& opened);
virtual ~Account();

/* getters */
string getAccountNumber() const { return accountNumber; }
string getAccountType() const { return accountType; }
double getBalance() const { return balance; }
string getDateOpened() const { return dateOpened; }
string getStatus() const { return status; }
double getTotalDeposits() const { return totalDeposits; }
double getTotalWithdrawals() const { return totalWithdrawals; }

/* setters / mutators */
void setBalance(double b) { balance = b; }
void setStatus(const string& s) { status = s; }
void addDeposit(double a) { balance += a; totalDeposits += a; }
void addWithdrawal(double a) { balance -= a; totalWithdrawals += a; }

/* pure virtual – each account type defines its own rules */
virtual double minimumDeposit() const = 0;
virtual double annualInterestRate() const = 0;
virtual double withdrawalLimit() const = 0; // per transaction (0 = no limit)
virtual int typeTag() const = 0; // 1..4 used for file storage
virtual string extraInfo() const { return ""; }

/* virtual behaviour */
```

```
virtual bool canWithdraw(double amount, string& reason) const;
virtual double calculateInterest() const;
virtual void display() const;

/* binary serialisation */
virtual void writeToFile(ofstream& out) const;
virtual void readFromFile(ifstream& in);
};
```

Four derived classes (show signatures only, Models.h)

```
class SavingsAccount : public Account {
private:
double interestRate; // % per annum
double minimumBalance;
public:
SavingsAccount();
SavingsAccount(const string& accNo, double initialDeposit, const string& opened);

double minimumDeposit() const { return 50.0; }
double annualInterestRate() const { return interestRate; }
double withdrawalLimit() const { return 5000.0; }
int typeTag() const { return 1; }

string extraInfo() const;
void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};

class ChequeAccount : public Account {
private:
double interestRate;
double overdraftLimit;
public:
ChequeAccount();
ChequeAccount(const string& accNo, double initialDeposit, const string& opened);

double minimumDeposit() const { return 100.0; }
double annualInterestRate() const { return interestRate; }
```

```
double withdrawalLimit() const { return 10000.0; }
int typeTag() const { return 2; }

bool canWithdraw(double amount, string& reason) const;
string extraInfo() const;
void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};

class FixedDepositAccount : public Account {
private:
double interestRate;
int termMonths;
string maturityDate;
public:
FixedDepositAccount();
FixedDepositAccount(const string& accNo, double initialDeposit, const string& opened);

double minimumDeposit() const { return 1000.0; }
double annualInterestRate() const { return interestRate; }
double withdrawalLimit() const { return 0.0; } // locked until maturity
int typeTag() const { return 3; }

bool canWithdraw(double amount, string& reason) const;
string extraInfo() const;
void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};

class StudentAccount : public Account {
private:
double interestRate;
double monthlyFee;
public:
StudentAccount();
StudentAccount(const string& accNo, double initialDeposit, const string& opened);

double minimumDeposit() const { return 20.0; }
double annualInterestRate() const { return interestRate; }
double withdrawalLimit() const { return 2000.0; }
```

```
int typeTag() const { return 4; }

string extraInfo() const;
void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};
```

Polymorphism in action (Models.cpp)

```
bool ChequeAccount::canWithdraw(double amount, string& reason) const {
if (amount <= 0) { reason = "Amount must be greater than zero."; return false; }

double lim = withdrawalLimit();
if (lim > 0 && amount > lim) {
stringstream ss;
ss << "Withdrawal exceeds the per-transaction limit of " << Utils::formatMoney(lim) << ".";
reason = ss.str();
return false;
}
if (amount > balance + overdraftLimit) {
reason = "Exceeds available balance plus the approved overdraft limit.";
return false;
}
return true;
}
```

1.3 Transaction Processing System

1.3.1 Transaction Authentication

File to reference: BankSystem.cpp (verifyCustomerPin)

```
bool BankSystem::verifyCustomerPin(Customer* cust) {
if (cust->isLocked()) {
cout << "\n [!] This account is currently LOCKED. Ask a supervisor to unlock it.\n";
return false;
}

int attempts = 0;
while (attempts < maxLoginAttempts) {
string pin = Utils::readLine(" Verify customer 5-digit PIN: ");
if (Utils::verifyText(pin, cust->getEncryptedPin())) {
cust->setFailedAttempts(0);
saveCustomers();
return true;
}
attempts++;
cout << " [!] Incorrect PIN. Attempts remaining: "
<< (maxLoginAttempts - attempts) << "\n";
}

cust->setLocked(true);
saveCustomers();
cout << "\n [!] Maximum attempts exceeded. Account has been LOCKED.\n";
return false;
}
```

1.3.2 Transaction Types

```
void BankSystem::recordTransaction(const string& accNo, const string& type, double amount,
double balanceAfter, const string& performedBy,
const string& branchCode, const string& desc) {
```

```
Transaction t(generateTransactionId(), accNo, type, amount, balanceAfter,
Utils::currentDateTime(), performedBy, branchCode, desc);
transactions.push_back(t);
saveTransactions();
saveConfiguration(); // persist the transaction counter
}
```

Withdrawal + Transfer

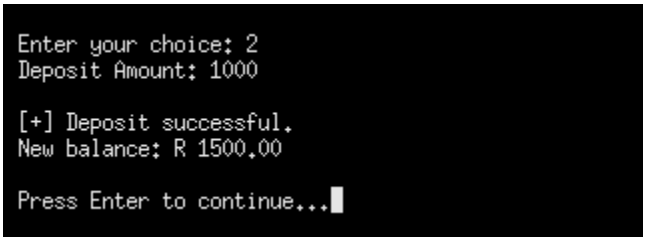
```
void BankSystem::doWithdrawal(Account* acc, double amount, const string& performedBy,
const string& branchCode) {
if (amount <= 0) throw BankingException("Withdrawal amount must be greater than zero.");

string reason;
if (!acc->canWithdraw(amount, reason)) throw BankingException(reason);

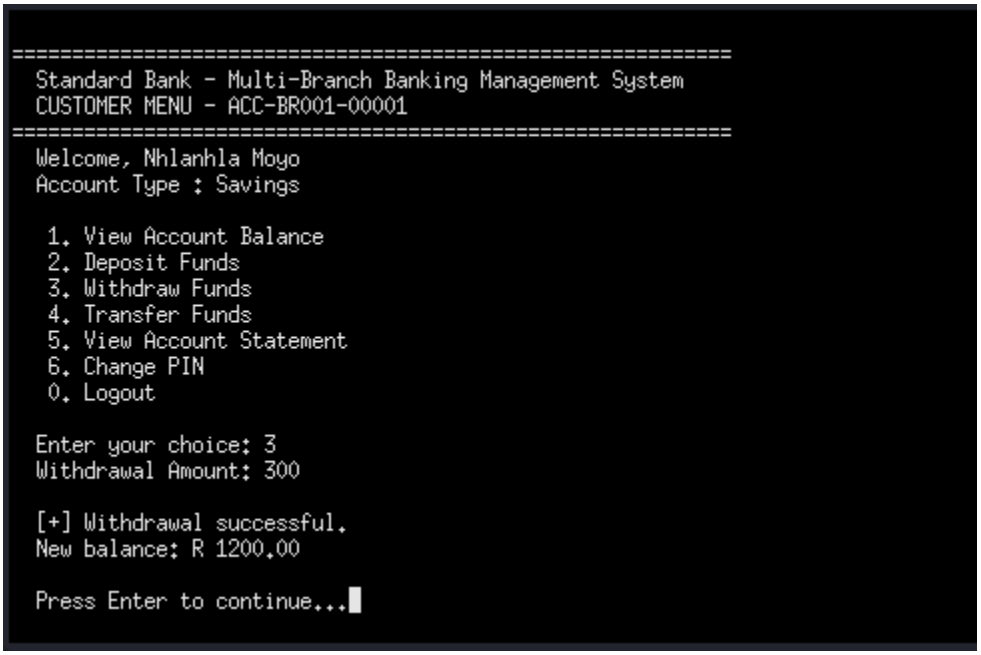
acc->addWithdrawal(amount);
recordTransaction(acc->getAccountNumber(), "Withdrawal", amount, acc->getBalance(),
performedBy, branchCode, "Cash withdrawal");
saveCustomers();

cout << "\n [+] Withdrawal successful.\n";
cout << " New balance: " << Utils::formatMoney(acc->getBalance()) << "\n";
}
```

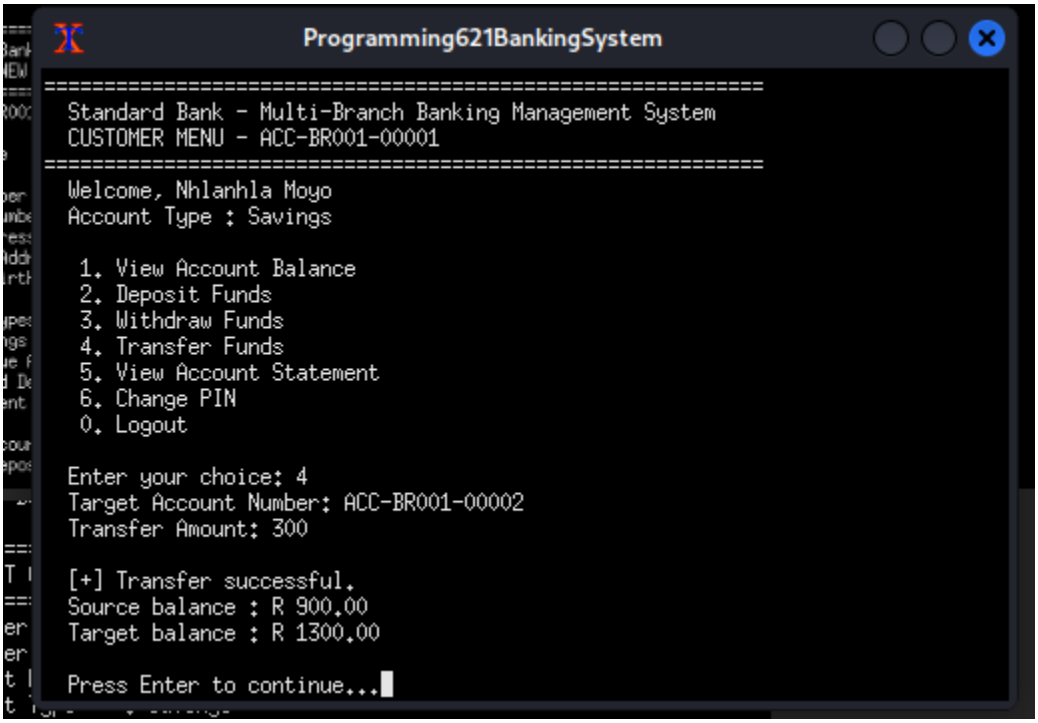
Cash Deposit



Cash Withdrawal



Fund Transfer



Account Statement

```
  3. Withdraw Funds
  4. Transfer Funds
  5. View Account Statement
  6. Change PIN
  0. Logout

Enter your choice: 5

============================================================
ACCOUNT STATEMENT - ACC-BR001-00001
Account Holder : Nhlanhla Moyo
Current Balance: R 900.00
============================================================

------------------------------------------------------------------------
Date/Time           Type                 Amount        Balance
------------------------------------------------------------------------
21/09/2026 10:12:32  Deposit              500.00         500.00
21/09/2026 11:47:55  Deposit             1000.00        1500.00
21/09/2026 11:50:05  Withdrawal           300.00        1200.00
21/09/2026 12:09:54  Transfer-Out         300.00         900.00
------------------------------------------------------------------------

Press Enter to continue...
```

1.3.3 Transaction Validation and Error Handling

Custom exception class

```
validation errors can be thrown and caught with try / catch. */
class BankingException : public std::runtime_error {
public:
explicit BankingException(const string& msg) : std::runtime_error(msg) {}
};
```

try/catch in the transaction processor

```
void BankSystem::processTellerTransaction() {
Utils::clearScreen();
printHeader("PROCESS CUSTOMER TRANSACTION");

string accNo = Utils::toUpper(Utils::readLine(" Customer Account Number: "));
Account* acc = 0;
Customer* cust = findCustomerByAccount(accNo, &acc);
```

```
if (!cust || !acc) {
cout << "\n [!] Account not found.\n";
Utils::pause();
return;
}

cout << "\n Customer : " << cust->getFullName() << "\n";
cout << " Account : " << acc->getAccountNumber()
<< " (" << acc->getAccountType() << ")\n";
cout << " Balance : " << Utils::formatMoney(acc->getBalance()) << "\n\n";

if (!verifyCustomerPin(cust)) { Utils::pause(); return; }

cout << "\n Transaction Types:\n";
cout << " 1. Cash Deposit\n";
cout << " 2. Cash Withdrawal\n";
cout << " 3. Fund Transfer\n";
cout << " 4. Balance Enquiry\n";
cout << " 5. Account Statement\n";
cout << " 0. Cancel\n";

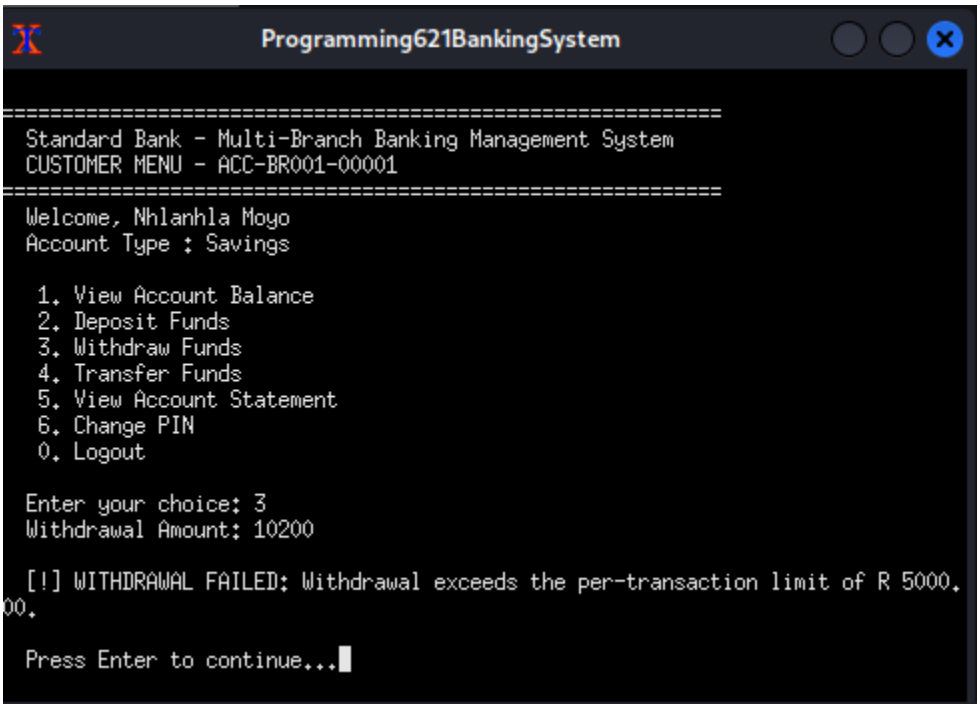
int t = Utils::readInt("\n Select transaction type: ");

try {
switch (t) {
case 1: {
double amt = Utils::readDouble(" Deposit Amount: ");
doDeposit(acc, amt, currentTeller->getTellerId(),
currentTeller->getBranchCode());
break;
}
case 2: {
double amt = Utils::readDouble(" Withdrawal Amount: ");
doWithdrawal(acc, amt, currentTeller->getTellerId(),
currentTeller->getBranchCode());
break;
}
case 3: {
string target = Utils::toUpper(Utils::readLine(" Target Account Number: "));
Account* to = findAccountAnywhere(target);
if (!to) throw BankingException("Target account not found.");
if (to == acc) throw BankingException("Cannot transfer to the same account.");
```

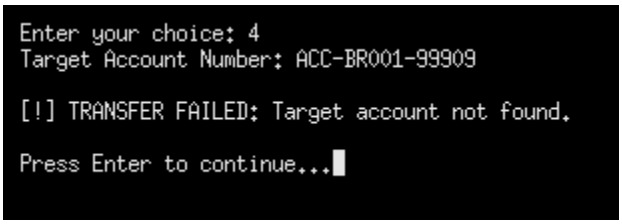
```
double amt = Utils::readDouble(" Transfer Amount: ");
doTransfer(acc, to, amt, currentTeller->getTellerId(),
currentTeller->getBranchCode());
break;
}
case 4:
cout << "\n Current Balance: " << Utils::formatMoney(acc->getBalance()) << "\n";
break;
case 5:
printStatement(acc->getAccountNumber());
break;
case 0:
cout << "\n Transaction cancelled.\n";
break;
default:
throw BankingException("Invalid transaction type selected.");
}
}
catch (const BankingException& ex) {
cout << "\n [!] TRANSACTION FAILED: " << ex.what() << "\n";
}
catch (const exception& ex) {
cout << "\n [!] Unexpected error: " << ex.what() << "\n";
}

Utils::pause();
}
```

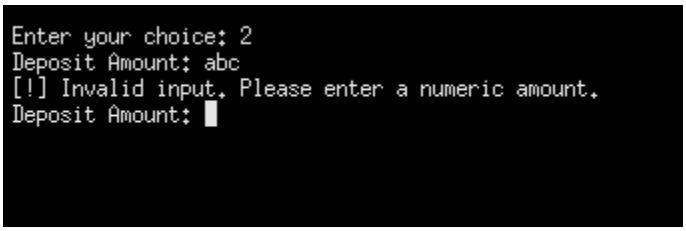
Insufficient funds



Target account not found



Invalid numeric input

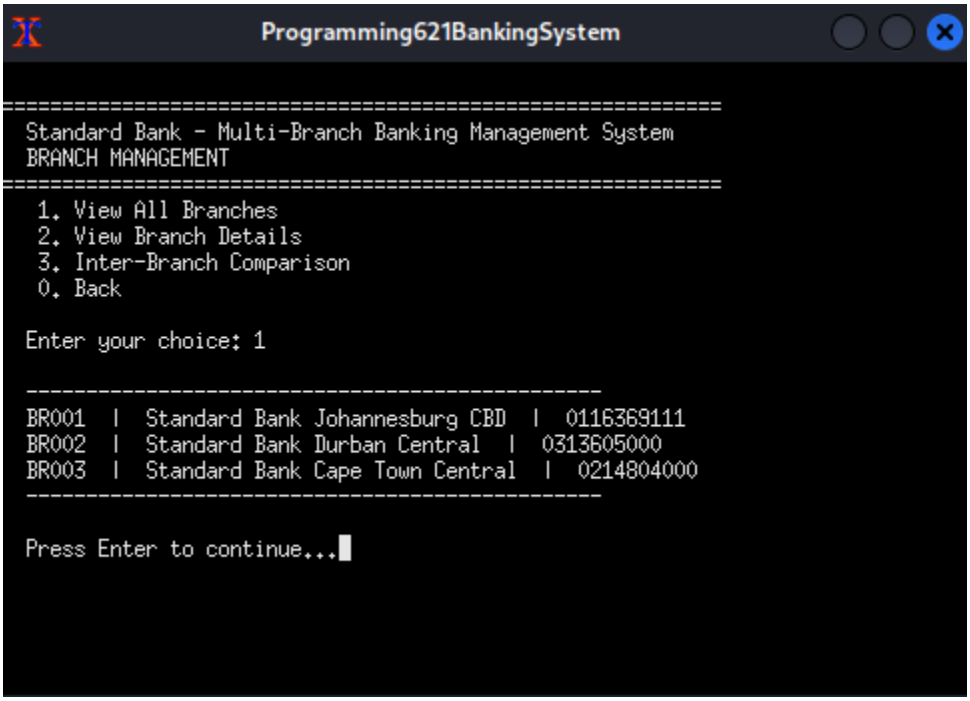


1.4 Multi-Branch Management

```
struct Branch {
string branchCode;
string branchName;
string address;
string contactNumber;
string managerName;

Branch();
Branch(const string& code, const string& name, const string& addr,
const string& contact, const string& mgr);

void display() const;
void writeToFile(ofstream& out) const;
void readFromFile(ifstream& in);
};
```



1.4.2 Branch Management Functions

key loop showing all three functions:

```
void BankSystem::branchMenu() {
while (true) {
Utils::clearScreen();
printHeader("BRANCH MANAGEMENT");
cout << " 1. View All Branches\n";
cout << " 2. View Branch Details\n";
cout << " 3. Inter-Branch Comparison\n";
cout << " 0. Back\n";

int choice = Utils::readInt("\n Enter your choice: ");

if (choice == 1) {
cout << "\n ------------------------------------------------\n";
for (size_t i = 0; i < branches.size(); ++i) {
cout << " " << branches[i].branchCode << " | "
<< branches[i].branchName << " | "
<< branches[i].contactNumber << "\n";
}
cout << " ------------------------------------------------\n";
Utils::pause();
}
else if (choice == 2) {
string code = Utils::toUpper(Utils::readLine(" Enter Branch Code (e.g. BR001): "));
Branch* b = findBranch(code);
if (!b) {
cout << "\n [!] Branch not found.\n";
} else {
cout << "\n ------------------------------------------------\n";
b->display();

/* statistics */
int accCount = 0; double totalBal = 0.0;
for (size_t i = 0; i < customers.size(); ++i) {
vector<Account*>& as = customers[i]->getAccounts();
for (size_t j = 0; j < as.size(); ++j) {
if (as[j]->getAccountNumber().substr(4, 5) == code) {
accCount++;
totalBal += as[j]->getBalance();
}
```

```
}
}
cout << " Accounts Held : " << accCount << "\n";
cout << " Total Deposits : " << Utils::formatMoney(totalBal) << "\n";
cout << " ------------------------------------------------\n";
}
Utils::pause();
}
else if (choice == 3) {
cout << "\n ---------------------------------------------------------------\n";
cout << " " << left << setw(10) << "Code"
<< setw(34) << "Branch Name"
<< right << setw(12) << "Accounts"
<< setw(18) << "Total Balance" << "\n";
cout << " ---------------------------------------------------------------\n";

for (size_t i = 0; i < branches.size(); ++i) {
string code = branches[i].branchCode;
int accCount = 0; double totalBal = 0.0;

for (size_t k = 0; k < customers.size(); ++k) {
vector<Account*>& as = customers[k]->getAccounts();
for (size_t j = 0; j < as.size(); ++j) {
if (as[j]->getAccountNumber().substr(4, 5) == code) {
accCount++;
totalBal += as[j]->getBalance();
}
}
}

cout << " " << left << setw(10) << code
<< setw(34) << branches[i].branchName
<< right << setw(12) << accCount
<< setw(18) << fixed << setprecision(2) << totalBal << "\n";
}
cout << " ---------------------------------------------------------------\n";
Utils::pause();
}
else if (choice == 0) {
return;
}
else {
cout << " [!] Invalid option.\n";
```

```
Utils::pause();
}
}
}
```

View Branch Details



Inter-Branch Comparison



1.5 Advanced Features

1.5.1 Interest Calculation and Application

Virtual interest method (Models.cpp)

```
double Account::calculateInterest() const {
/* simple annual interest = balance * rate / 100 */
return balance * annualInterestRate() / 100.0;
}
```

Apply interest to all accounts (BankSystem.cpp)

```
void BankSystem::applyInterest() {

Utils::clearScreen();
printHeader("APPLY INTEREST TO ELIGIBLE ACCOUNTS");

cout << " This will apply the annual interest rate to every active account.\n";
string confirm = Utils::readLine(" Type YES to continue: ");
if (Utils::toUpper(confirm) != "YES") {
cout << "\n Operation cancelled.\n";
Utils::pause();
return;
}

int applied = 0;
double totalInterest = 0.0;

cout << "\n ----------------------------------------------------------------\n";
cout << " " << left << setw(22) << "Account"
<< setw(18) << "Type"
<< right << setw(14) << "Balance"
<< setw(14) << "Interest" << "\n";
cout << " ----------------------------------------------------------------\n";

for (size_t i = 0; i < customers.size(); ++i) {
vector<Account*>& as = customers[i]->getAccounts();
for (size_t j = 0; j < as.size(); ++j) {
Account* a = as[j];
if (a->getStatus() != "Active") continue;
```

```
double interest = a->calculateInterest();
if (interest <= 0.0) continue;

a->addDeposit(interest);
recordTransaction(a->getAccountNumber(), "Interest", interest,
a->getBalance(), currentTeller->getTellerId(),
currentTeller->getBranchCode(),
"Annual interest applied at "
+ to_string(a->annualInterestRate()) + "% p.a.");

cout << " " << left << setw(22) << a->getAccountNumber()
<< setw(18) << a->getAccountType()
<< right << setw(14) << fixed << setprecision(2) << (a->getBalance() - interest)
<< setw(14) << interest << "\n";

applied++;
totalInterest += interest;
}
}

saveCustomers();
cout << " ------------------------------------------------------------------\n";
cout << " Accounts credited : " << applied << "\n";
cout << " Total interest : " << Utils::formatMoney(totalInterest) << "\n";
Utils::pause();
}
```

1.5.2 Customer Search and Filtering

```
void BankSystem::searchCustomersMenu() {
while (true) {
Utils::clearScreen();
printHeader("CUSTOMER SEARCH AND FILTERING");
cout << " 1. Search by Account Number\n";
cout << " 2. Search by SA ID Number\n";
cout << " 3. Search by Surname / Name (partial match)\n";
cout << " 4. Filter by Account Type\n";
cout << " 5. Filter by Minimum Balance\n";
cout << " 0. Back\n";
```

```
int choice = Utils::readInt("\n Enter your choice: ");
if (choice == 0) return;

int results = 0;

switch (choice) {
case 1: {
string q = Utils::toUpper(Utils::readLine(" Account Number: "));
Account* a = 0;
Customer* c = findCustomerByAccount(q, &a);
if (c && a) {
cout << "\n Match found:\n";
c->display();
cout << "\n Account details:\n";
a->display();
results = 1;
}
break;
}
case 2: {
string q = Utils::readLine(" SA ID Number: ");
Customer* c = findCustomerByIdNumber(q);
if (c) {
cout << "\n Match found:\n";
c->display();
results = 1;
}
break;
}
case 3: {
string q = Utils::toUpper(Utils::readLine(" Enter name or surname: "));
cout << "\n Results:\n";
for (size_t i = 0; i < customers.size(); ++i) {
string full = Utils::toUpper(customers[i]->getFullName());
if (full.find(q) != string::npos) {
cout << " ----------------------------------------\n";
customers[i]->display();
results++;
}
}
break;
}
```

```
case 4: {
cout << "\n Account Types: 1=Savings 2=Cheque 3=Fixed Deposit 4=Student\n";
string t = Utils::readLine(" Enter type name or number: ");
string typeName;
if (t == "1") typeName = "Savings";
else if (t == "2") typeName = "Cheque";
else if (t == "3") typeName = "Fixed Deposit";
else if (t == "4") typeName = "Student";
else typeName = t;

cout << "\n Results:\n";
for (size_t i = 0; i < customers.size(); ++i) {
vector<Account*>& as = customers[i]->getAccounts();
for (size_t j = 0; j < as.size(); ++j) {
if (as[j]->getAccountType() == typeName) {
cout << " ----------------------------------------\n";
cout << " " << customers[i]->getFullName()
<< " | " << as[j]->getAccountNumber()
<< " | " << Utils::formatMoney(as[j]->getBalance()) << "\n";
results++;
}
}
}
break;
}
case 5: {
double minBal = Utils::readDouble(" Minimum Balance (R): ");
cout << "\n Results:\n";
for (size_t i = 0; i < customers.size(); ++i) {
vector<Account*>& as = customers[i]->getAccounts();
for (size_t j = 0; j < as.size(); ++j) {
if (as[j]->getBalance() >= minBal) {
cout << " ----------------------------------------\n";
cout << " " << customers[i]->getFullName()
<< " | " << as[j]->getAccountNumber()
<< " | " << as[j]->getAccountType()
<< " | " << Utils::formatMoney(as[j]->getBalance()) << "\n";
results++;
}
}
}
break;
}
```

```
default:
cout << " [!] Invalid option.\n";
}

cout << "\n Total matches found: " << results << "\n";
Utils::pause();
}
}
```

Search by SA ID

```
Enter your choice: 2
SA ID Number: 8001015009087

Match found:
Customer ID    : CUST-0001
Full Name      : Nhlanhla Moyo
SA ID Number   : 8001015009087
Contact Number : 0712345678
Email Address  : nhlanhla@example.com
Physical Addr. : 12 location street, JHB
Date of Birth  : 01/01/2000
Account Status : ACTIVE
Total Balance  : R 912.00
```

Partial name search (linear search)

```
Results:
--------------------------------
Customer ID    : CUST-0001
Full Name      : Nhlanhla Moyo
```

Filter by minimum balance

```
Enter your choice: 5
Minimum Balance (R): 1000

Results:
----------------------------------------
Programming C++ | ACC-BR001-00002 | Cheque | R 1300.00

Total matches found: 1

Press Enter to continue...
```

1.6 Reporting and Analytics

report loops (one snippet per report)

```
void BankSystem::dailyTransactionReport() {
Utils::clearScreen();
printHeader("DAILY TRANSACTION REPORT");

string today = Utils::currentDate();
cout << " Date: " << today << "\n\n";

int count = 0;
double totalDeposits = 0.0, totalWithdrawals = 0.0, totalTransfers = 0.0;

cout << " ------------------------------------------------------------------------\n";
cout << " " << left << setw(14) << "Txn ID"
<< setw(22) << "Account"
<< setw(15) << "Type"
<< right << setw(14) << "Amount"
<< " " << left << setw(8) << "By" << "\n";
cout << " ------------------------------------------------------------------------\n";

for (size_t i = 0; i < transactions.size(); ++i) {
if (transactions[i].dateTime.substr(0, 10) != today) continue;
if (transactions[i].branchCode != currentTeller->getBranchCode()) continue;

const Transaction& t = transactions[i];
cout << " " << left << setw(14) << t.transactionId
<< setw(22) << t.accountNumber
<< setw(15) << t.type
<< right << setw(14) << fixed << setprecision(2) << t.amount
<< " " << left << setw(8) << t.performedBy << "\n";

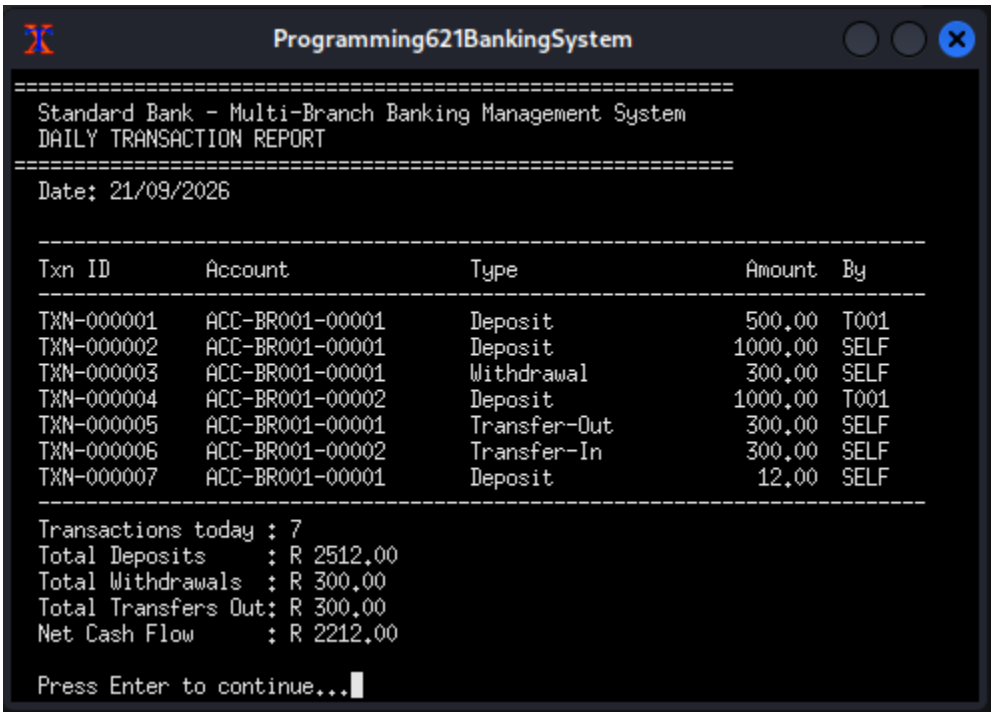
if (t.type == "Deposit") totalDeposits += t.amount;
if (t.type == "Withdrawal") totalWithdrawals += t.amount;
if (t.type == "Transfer-Out") totalTransfers += t.amount;

count++;
}

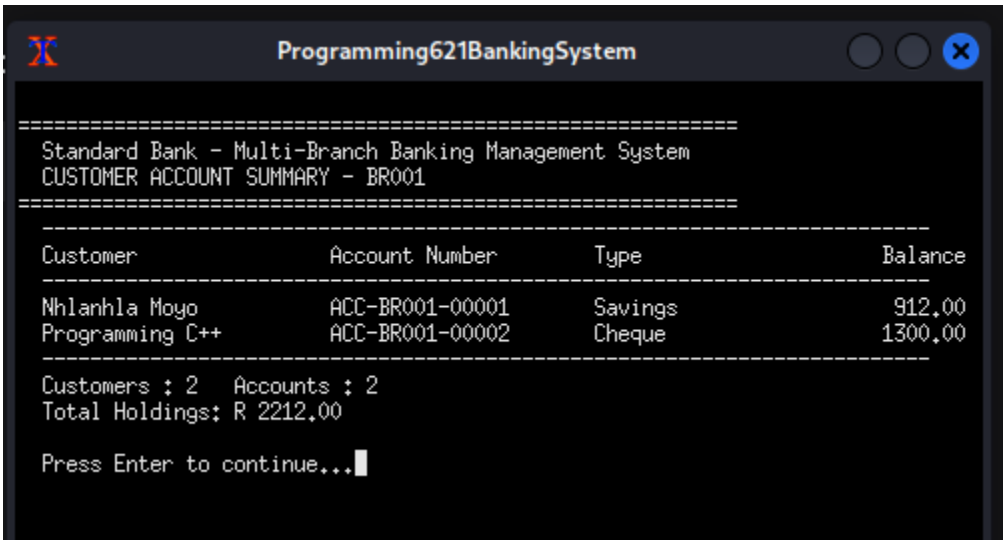
cout << " ------------------------------------------------------------------------\n";
cout << " Transactions today : " << count << "\n";
cout << " Total Deposits : " << Utils::formatMoney(totalDeposits) << "\n";
cout << " Total Withdrawals : " << Utils::formatMoney(totalWithdrawals) << "\n";
```

```
cout << " Total Transfers Out: " << Utils::formatMoney(totalTransfers) << "\n";
cout << " Net Cash Flow : "
<< Utils::formatMoney(totalDeposits - totalWithdrawals) << "\n";
Utils::pause();
}
```

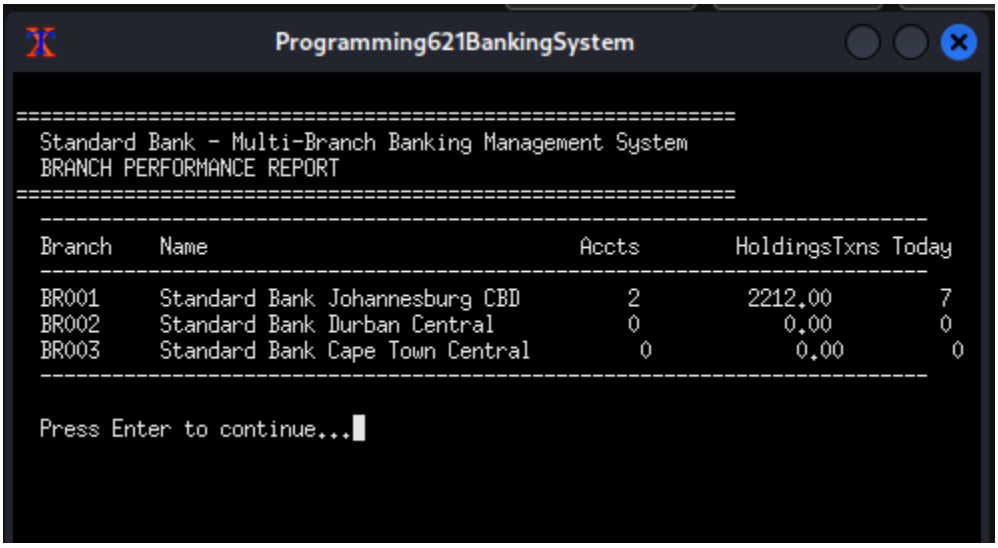
Daily Transaction Report



Customer Account Summary

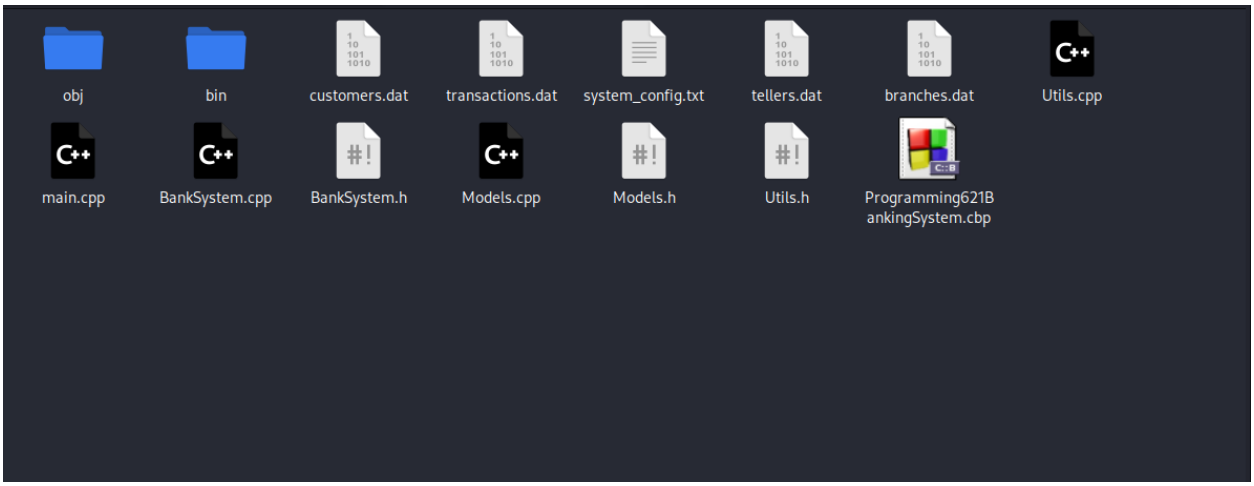


Branch Performance Report

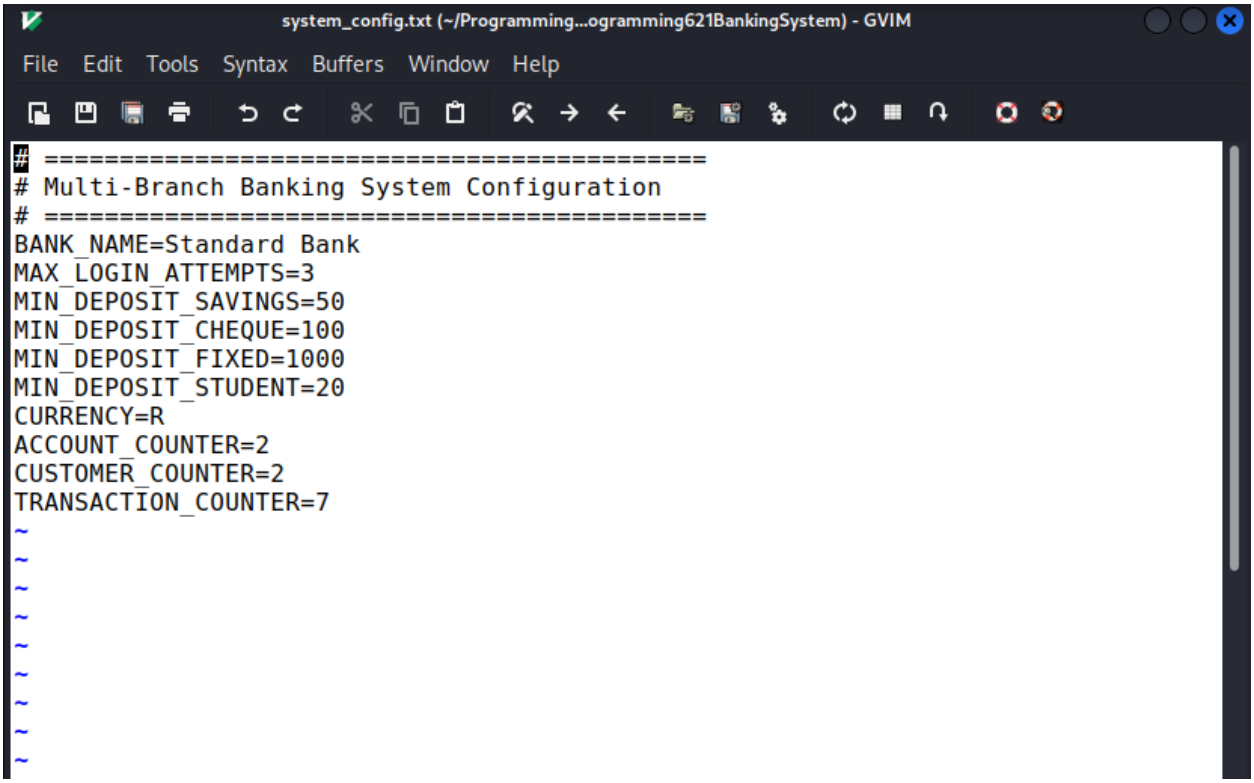


1.7 File Handling Requirements

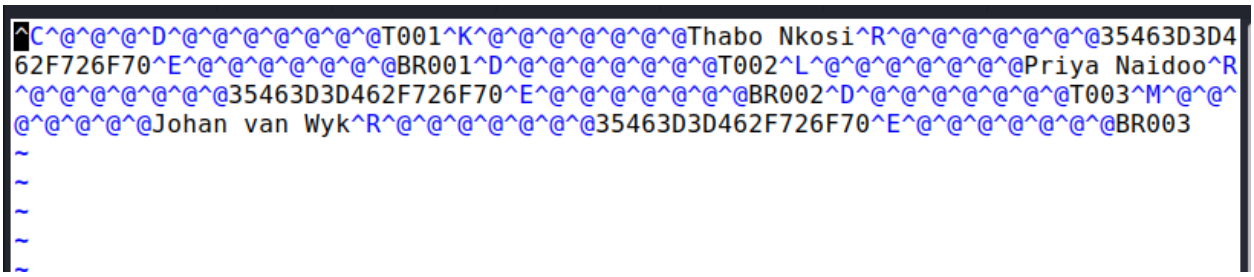
Binary string write/read helpers (Utils.cpp)



system_config.txt



tellers.dat



Backup and recovery

```
void BankSystem::backupData() {
Utils::clearScreen();
printHeader("BACKUP AND RECOVERY");

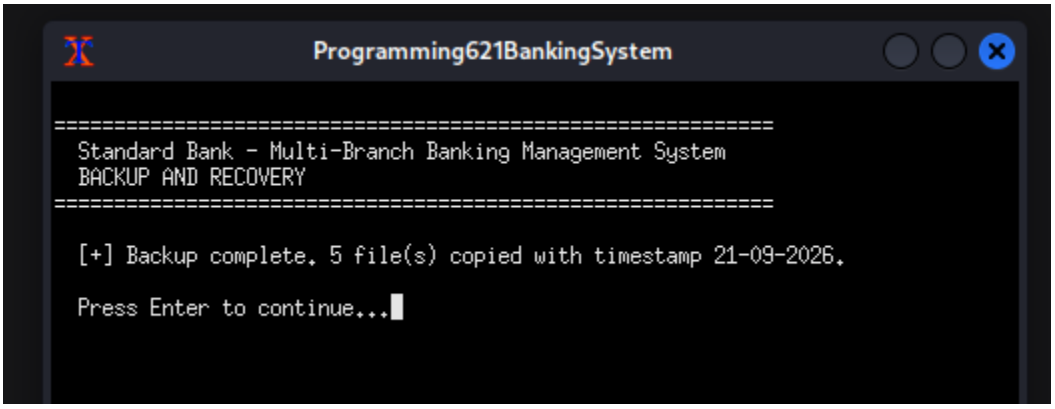
saveAll();

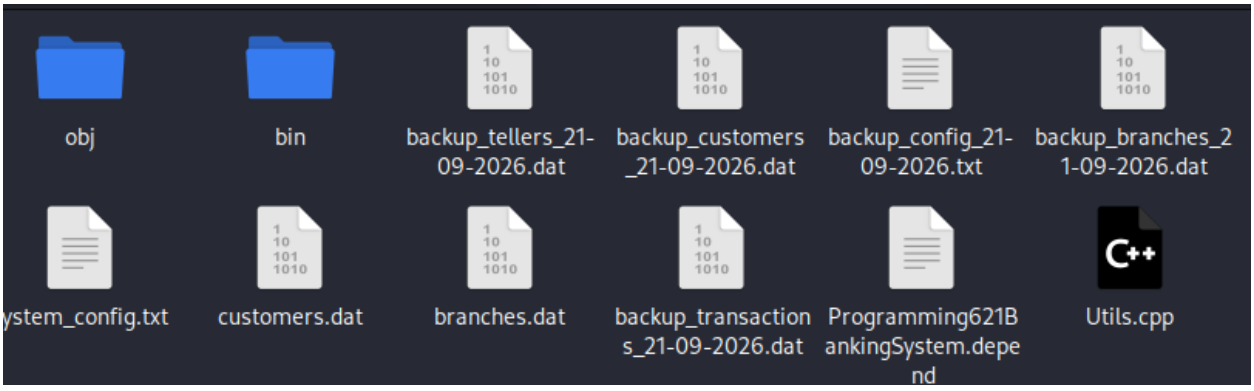
string stamp = Utils::currentDate();
/* replace slashes so the name is filesystem friendly */
for (size_t i = 0; i < stamp.size(); ++i) if (stamp[i] == '/') stamp[i] = '-';
```

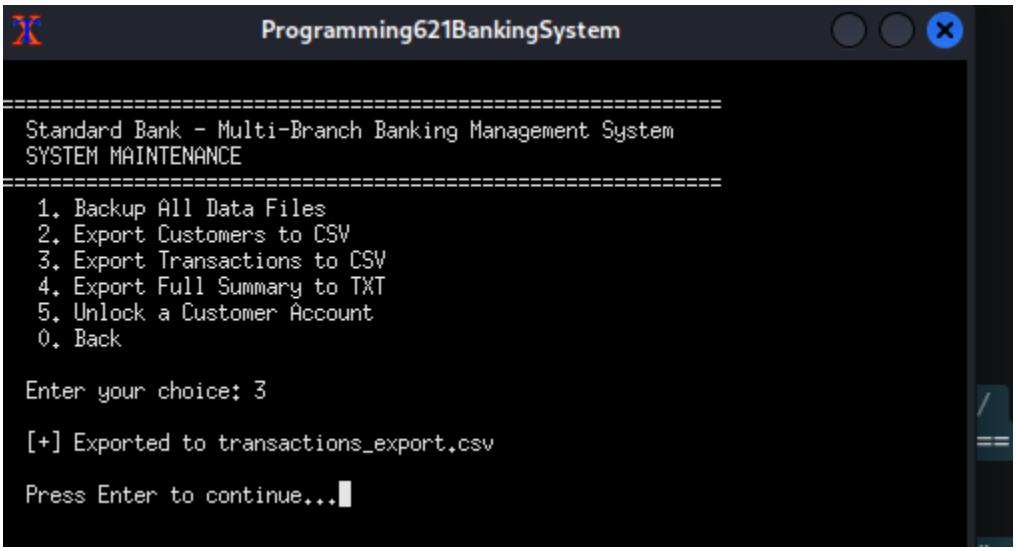
```
int ok = 0;
if (copyFileBinary(fileCustomers, "backup_customers_" + stamp + ".dat")) ok++;
if (copyFileBinary(fileTransactions, "backup_transactions_" + stamp + ".dat")) ok++;
if (copyFileBinary(fileBranches, "backup_branches_" + stamp + ".dat")) ok++;
if (copyFileBinary(fileTellers, "backup_tellers_" + stamp + ".dat")) ok++;
if (copyFileBinary(fileConfig, "backup_config_" + stamp + ".txt")) ok++;

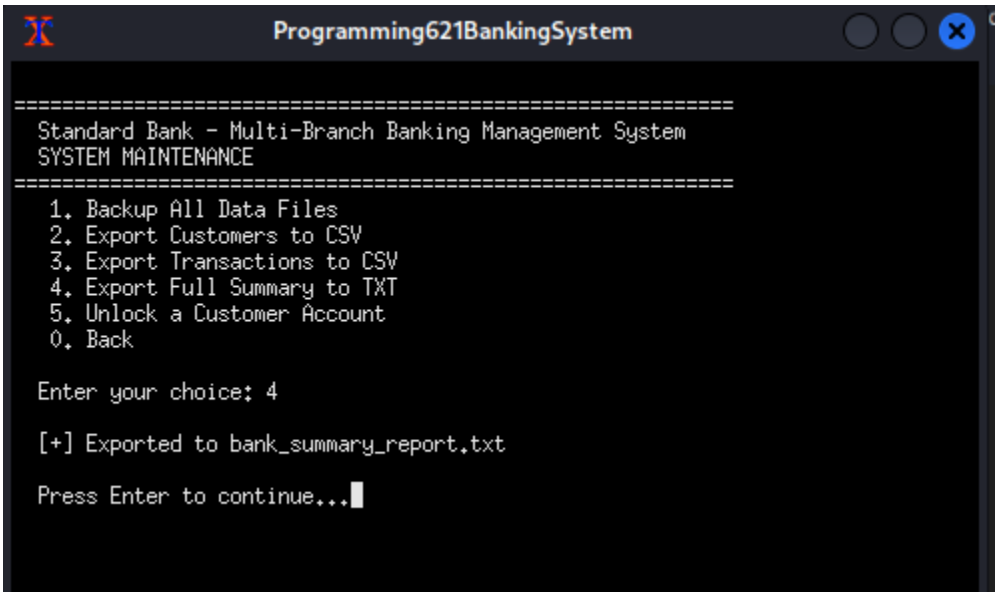
cout << "\n [+] Backup complete. " << ok << " file(s) copied with timestamp "
<< stamp << ".\n";
Utils::pause();
}
```

STEP 6 — Run the BACKUP feature

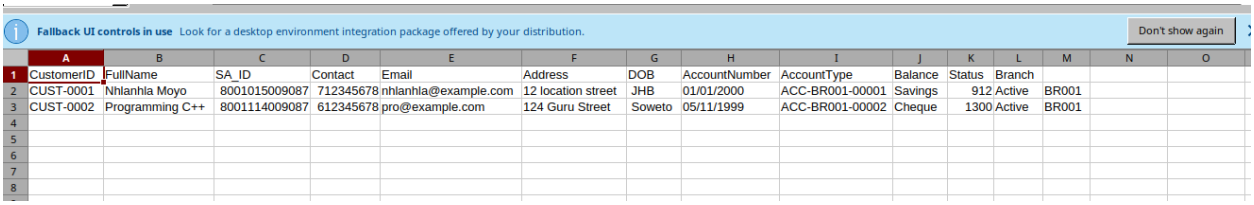





Programming621BankingSystem
Standard Bank - Multi-Branch Banking Management System
SYSTEM MAINTENANCE
1. Backup All Data Files
2. Export Customers to CSV
3. Export Transactions to CSV
4. Export Full Summary to TXT
5. Unlock a Customer Account
0. Back
Enter your choice: 3
[+] Exported to transactions_export.csv
Press Enter to continue...



customers_export.csv



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	CustomerID	FullName	SA_ID	Contact	Email	Address	DOB	AccountNumber	AccountType	Balance	Status	Branch			
2	CUST-0001	Nhlanhla Moyo	8001015009087	712345678	nhlanhla@example.com	12 location street	JHB	01/01/2000	ACC-BR001-00001	Savings	912	Active	BR001		
3	CUST-0002	Programming C++	8001114009087	612345678	pro@example.com	124 Guru Street	Soweto	05/11/1999	ACC-BR001-00002	Cheque	1300	Active	BR001		
4															
5															
6															
7															
8															

Bank Summery report

